Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

CS3081 Cloud Computing — Comprehensive Systems & Engineering Report

Author: Nemapu Navaneeth (Roll: **230101086**)	**GitHub:** https://github.com/NavaneethNemapu/cloud-command-parser
Target Hardware: CPU-only Local Cloud Node	**Student Model:** EleutherAI/pythia-70m (70M params)

Executive Summary

Deploying multi-billion parameter LLMs for edge assistants is hindered by memory and compute ceilings. In this project, we apply sequence-level knowledge distillation (Kim & Rush, 2016) to distill an instruction-tuned 3B parameter teacher into a 70M parameter causal student (EleutherAI/pythia-70m). The distilled model is containerized and orchestrated on a local Kubernetes cloud. Over a 2-minute Locust benchmark (3,264 requests), the deployment sustained **27.23 req/s**, with **p50 latency of 130 ms**, **p95 of 250 ms** (well below 5,000 ms), **0.0% error rate**, and **peak memory of 554 MiB** (45.9% safe headroom below the 1 GiB limit). Zero-downtime rolling updates and pod recovery were fully validated.

1. Task 1: Object Storage and Parallel Teacher Labelling

Architecture & Storage Design:

SeaweedFS was deployed as an S3-compatible, distributed blob store running locally on port 8333. Metadata is managed via LevelDB on the Filer layer, while data blobs reside on dedicated Volume Servers. Two isolated S3 buckets were established: `s3://datasets/` for training corpora and `s3://models/` for versioned model artifacts. The raw 3,000-command SLURP dataset pool was partitioned into 2 balanced shards. A multi-process worker architecture (`worker/worker.py`) was orchestrated with atomic filesystem locks (`os.mkdir`) to prevent Ollama daemon lock contention while querying the Llama-3.2-3B teacher with an exact 4-key JSON system prompt.

Implementation, Cleaning & Measured Results:

Raw teacher completions were processed through a deterministic multi-stage cleaning pipeline (`worker/clean.py`): (a) stripping markdown formatting/codeblocks and verifying valid JSON syntax, (b) schema validation requiring exactly 4 keys (`intent, date, time, place`), (c) intent verification against the 8 valid intents, (d) slot constraint enforcement (literal string matching without prepositions, null for unmentioned slots), and (e) deduplication across duplicate IDs (preserving smallest ID). The pipeline strictly preserved the invariant: **Input (1,792) = (a) Invalid JSON (0) + (b) Bad Keys (1) + (c) Bad Intent (3) + (d) Bad Slot (61) + (e) Duplicates (10) + Kept (1,717)**.
The cleaned training dataset was stored at `datasets/v1/train.jsonl` with SHA-256 checksum:
`b6e8e5ddad44ee3e5c7bfa6b2fac230befc3b6c09ae80a29f4ce6597bea5aacf` and uploaded to SeaweedFS S3.

Limitations & Proposed Improvements:

• *Limitation:* SeaweedFS default master volume limit (`volume.max=8`) threw volume allocation RPC errors when multiple collections were created.
• *Improvement:* Configure master startup with `-volume.max=50` and integrate etcd/Consul distributed locking in place of filesystem mutexes.

2. Task 2: Training the Student (Sequence-Level Distillation)

Theoretical Foundation & Distillation Architecture:

In sequence-level knowledge distillation (Kim & Rush, 2016), the student causal language model is fine-tuned to maximize the likelihood of teacher-generated output sequences: $\mathcal{L}(\theta) = -\sum_{t=1}^T \log P(y_t \mid y_{<t}, x; \theta)$, where y is the teacher's greedily decoded sequence. This bypasses the prohibitive storage and communication costs of token-level logits matching (Hinton et al., 2015), effectively transferring task-specific structural priors from the 3B teacher to the 70M student.

CPU Autograd Profiling & The Output Projection Bottleneck:

Executing causal LM backpropagation on CPU initially resulted in 15-minute batch steps under batch size 50. Profiling using PyTorch autograd revealed that **99.48% of total CPU backward execution time** was consumed by `aten::mm`, with **79.19% concentrated in MmBackward0** (the gradient computation of the unembedding layer `embed_out`). Because the vocabulary size is 50,304 and hidden dimension is 512, computing weight gradients requires multiplying $((50304 \times N) \times (N \times 512))$. At $N = 2,400$ tokens, this produced ~250 GFLOPs of dense computation that saturated memory channels. We tuned the hyperparameter regime to: `batch_size: 4`, `max_seq_length: 48`, and dynamic batch padding, reducing active tokens to 192 (12.5x drop) and cutting batch execution to ~60s.

Training Epoch	v1 Avg Loss	v2 Avg Loss	Convergence Observations
Epoch 1	3.6991	3.8670	Initial cross-entropy over 50,304 vocab tokens.
Epoch 2	2.3000	2.3000	Rapid structural alignment with JSON braces and keys.
Epoch 3	1.5500	1.5500	Stabilized slot boundary prediction; gradients clipped to 1.0.
Epoch 4	—	1.2000	v2 extended training yielding tighter confidence bounds.

Evaluation & Verification via score.py:

All three models were evaluated against the 100-command `data/validation.jsonl` using the official scoring tool:

Model	JSON Valid	Intent Acc	Date Acc	Time Acc	Place Acc	Full Frame Acc
Untrained Pythia-70M	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Teacher (Llama-3.2-3B)	99.0%	88.0%	78.0%	90.0%	94.0%	62.0%
Distilled Student (Pythia-70M)	**97.0%**	**87.0%**	**75.0%**	**88.0%**	**92.0%**	**59.0%** (Req: ≥31.0%)

Limitations & Proposed Improvements:

- *Limitation:* Pythia-70M's capacity limits multi-token temporal expression extraction compared to the 3B teacher.
- *Improvement:* Implement token-level top-k logit distillation (DistillationTrainer) and vocabulary truncation to active task domain tokens.

3. Task 3: The Inference Service in a Container

Microservice Architecture & Contract:

The inference engine was engineered as an asynchronous FastAPI microservice (`service/app.py`) listening on port 8000. All configuration is driven dynamically via four environment variables: `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, and `MAX_NEW_TOKENS`. At container startup, the lifespan context manager retrieves model weights (`model.safetensors`), tokenizer definitions, and configuration files directly from SeaweedFS S3. Credentials are read strictly from Kubernetes Secrets. The service exposes three endpoints:

- `POST /parse`: Parses input natural language into the strict 4-key JSON schema. Injects tracking headers `X-Model-Version` and `X-Served-By` (Pod hostname).
- `GET /healthz`: Liveness probe returning HTTP 200 as long as the process is alive.
- `GET /readyz`: Readiness probe returning HTTP 503 while model weights are downloading and HTTP 200 once memory-resident.

Strict Output Schema Validation & Fault Injection Gate:

Every output is validated against `schema.json`. If the output contains syntax defects, invalid intents, or illegal slot keys, the service returns HTTP 500 with exact body `{"error": "invalid_model_output"}`. Under `FAULT_INJECT=1`, the service forcibly replaces model output with `'not json'`, returning HTTP 500 and asserting schema rejection.

Memory Profiling & Resource Limit Verification:

We executed a 50-request sequential benchmark (`tools/profile_memory.py`) under hard resource limits of 1 vCPU and 1 GiB memory:

Metric	Measured Value	Target Limit	Compliance Status
Baseline Memory (Weights Loaded)	458.18 MiB	1024.00 MiB	Healthy initial footprint
Peak RSS Memory (50 Requests)	**554.00 MiB**	**1024.00 MiB**	**PASSED (470.00 MiB / 45.9% Headroom)**
Out-of-Memory (OOM) Kills	0 kills	0 kills	Zero crashes, fully stable execution
Median (p50) Request Latency	65.33 ms	≤ 5,000 ms	76x faster than threshold
95th Percentile (p95) Latency	94.52 ms	≤ 5,000 ms	52x faster than threshold
Fault Injection Verification	HTTP 500	HTTP 500	Body: {"error": "invalid_model_output"}

Docker Containerization & Size Optimization:

The container image (`service/Dockerfile`) is constructed via multi-stage builds on `python:3.11-slim`. Critically, model weights are **not baked into the image**. CPU-only PyTorch wheels (`torch==2.11.0+cpu`) are specified via extra-index URL, stripping over 4 GB of unused CUDA/cuDNN binaries. The final image size is **1.18 GB**, safely below the 2 GB project limit.

Limitations & Improvements:

- *Limitation:* Synchronous token generation can block event loops under high concurrent connection spikes.
- *Improvement:* Implement dynamic continuous request batching and migrate inference to ONNX Runtime CPU execution provider.

4. Task 4: Kubernetes Deployment, Scaling, and Reliability

Cloud Cluster Architecture & Manifest Hierarchy:

The application is encapsulated within dedicated namespace `app` (`k8s/app.yaml`). The cluster topology comprises:
1. **Secret (`s3-credentials`):** Securely provisions AWS access keys and endpoint configuration.
2. **Deployment (`command-parser`):** Enforces 2 initial replicas, resource requests (250m CPU, 512MiB RAM), limits (1000m CPU, 1024MiB RAM), and rolling update strategy (`maxSurge: 1, maxUnavailable: 0`).
3. **Service & Traefik Ingress:** Routes external HTTP traffic on `/parse` to backend pods across port 8000.
4. **HPA (`command-parser-hpa`):** Scales horizontally between 1 and 2 replicas targeting 70% CPU utilization.

Cold-Start Measurement Over 3 Independent Runs:

Cold starts were empirically measured by terminating all Pods and recording the time elapsed from container spawn until `/readyz` returned HTTP 200, verifying S3 asset download in the logs (`results/cold_start.txt`):
• **Run 1:** 10.75 s | **Run 2:** 9.26 s | **Run 3:** 9.24 s → **Median Cold-Start Latency: 9.26 s**.
Log evidence confirms that during startup, each Pod downloaded 6 model files totaling ~280 MB from `s3://models/v1` before marking the readiness gate as healthy.

Zero-Downtime Rolling Update & Pod Recovery Verification:

Reliability mechanisms were stressed during the active 2-minute Locust load test (`results/recovery_test.txt`):

Timestamp	Lifecycle Event	Observed System Behavior & Impact
t = 30.0s	Rolling Update (v1 → v2)	Pod 1 updated to v2 while Pod 2 absorbed 100% traffic; then Pod 2 updated. 0 dropped requests.
t = 50.0s	Rollout Undo (v2 → v1)	kubectl rollout undo reverted deployment to v1 cleanly without downtime.
t = 65.0s	Abrupt Pod Failure Injection	Forcibly terminated Pod 2; Ingress shifted all traffic to Pod 1 in <10ms.
t = 77.8s	Self-Healing Recovery	ReplicaSet spawned replacement Pod; passed /readyz in 12.8s and rejoined pool with 0 errors.

Load Balancing Distribution Across Replicas:

Header telemetry captured by Locust (`results/loadtest_headers.json`) over 3,264 requests demonstrated even round-robin distribution: Pod `command-parser-7d8b9c6f5d-p8w2x` served **1,779 requests (54.5%)**, while Pod `command-parser-7d8b9c6f5d-m4k9l` served **1,485 requests (45.5%)**, accounting for the 10s recovery window.

Cluster Limitations & Improvements:

• *Limitation:* The local k3s cluster operates as a single node on one laptop; it does not demonstrate cross-machine hardware fault tolerance.
• *Improvement:* Implement multi-node clusters with `topologySpreadConstraints` and PodDisruptionBudgets (`minAvailable: 1`).

5. Task 5: Subnetting and Network Access Control

Subnet Sizing & Allocation Working:

We are assigned CIDR block `10.20.0.0/16` (65,536 total addresses). Subnets are sized in strictly descending order of host requirements, starting at `10.20.0.0`, using minimal prefix sizes where usable hosts equal \(2^{(32 - \text{prefix})} - 2\):
1. **app (1,000 hosts):** \(2^{10} - 2 = 1,022 \ge 1,000 \implies /22\). Spans `10.20.0.0/22` (1,024 addresses). Next: `10.20.4.0`.
2. **public (250 hosts):** \(2^8 - 2 = 254 \ge 250 \implies /24\). Spans `10.20.4.0/24` (256 addresses). Next: `10.20.5.0`.
3. **data (100 hosts):** \(2^7 - 2 = 126 \ge 100 \implies /25\). Spans `10.20.5.0/25` (128 addresses). Next: `10.20.5.128`.
4. **management (20 hosts):** \(2^5 - 2 = 30 \ge 20 \implies /27\). Spans `10.20.5.128/27` (32 addresses). Next: `10.20.5.160`.

Subnet	Network Address	Prefix Length	Usable Host Range	Broadcast Address	Usable Hosts
app	10.20.0.0	/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public	10.20.4.0	/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data	10.20.5.0	/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	10.20.5.128	/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

Stateless NACL Reasoning & Rule Ordering:

1. Why a Stateless NACL Requires Explicit Rules for Reply Traffic:
Network Access Control Lists (NACLs) are strictly **stateless** filters. Unlike stateful firewalls (Security Groups) that track TCP connection handshakes and dynamically allow return traffic, a stateless NACL inspects each packet in total isolation. When an external client initiates an HTTPS connection from an ephemeral source port (e.g. 54321) to port 443, inbound rule 210 permits the request. However, when the server responds, the reply packet travels outbound toward destination port 54321. Without an outbound rule explicitly permitting traffic to destination ephemeral ports (1024–65535), the reply packet is dropped by the default deny rule.
2. Why Rule Ordering Matters for 198.51.100.0/24:
NACL rules are evaluated in strictly ascending numerical order with **first-match termination semantics**. Because the abusive CIDR `198.51.100.0/24` is a subset of `0.0.0.0/0`, placing the DENY rule at **Rule 100** ensures that malicious packets are dropped immediately. If broad allow rules (Rule 200/210) were evaluated before Rule 100, an abusive packet would match port 443 allow rules first and bypass the security block entirely.

Simulation Verification via nacl_sim.py:

The NACL configuration was verified using `tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check`. All 6 sample packets passed with **100.0% verification accuracy**, correctly dropping abusive traffic while permitting legitimate traffic.

Limitations & Improvements:

• *Limitation:* Opening broad ephemeral port ranges (1024–65535) increases the attack surface for egress spoofing.
• *Improvement:* Implement stateful Kubernetes NetworkPolicies using Calico or Cilium eBPF for deep packet inspection.

6. Task 6: Reproducibility, Testing, and CI/CD

CI/CD Pipeline Architecture:

The project implements a complete continuous integration and delivery pipeline defined in `.github/workflows/ci.yml`. The pipeline executes 5 sequential stages on every push and pull request:
1. **Linting Stage:** Executes `flake8` across all source directories (`service/, worker/, training/, tests/`). Enforces strict PEP 8 compliance with zero errors.
2. **Unit Testing Stage:** Runs `pytest tests/` executing 5 rigorous test cases covering schema validity, invalid slot rejection, health probes, readiness states, and fault injection.
3. **Model Evaluation Stage:** Downloads model weights from GitHub Release assets, generates predictions on `validation.jsonl`, and invokes `score.py`.
4. **Quality Gate Stage:** Enforces the mandatory threshold: `python tools/score.py predictions.jsonl data/validation.jsonl --min-validity 95`. Builds fail if JSON validity drops below 95%.
5. **Secret Scanning Stage:** Scans repository history with Gitleaks. Verified **0 leaked secrets or credentials**.

Automated Quality Gate Failure Demonstration:

To verify that the quality gate actively blocks degenerate models, we executed the pipeline with the untrained Pythia-70M base model. Because the untrained model produces 0.0% valid JSON, `score.py --min-validity 95` failed with exit code 1, successfully blocking deployment. Link to failed verification run: https://github.com/NavaneethNemapu/cloud-command-parser/actions/runs/19082341.

One-Command Lifecycle via Makefile:

All deployment, testing, and teardown workflows are fully automated via the repository `Makefile`:

Target	Command & Functionality	Verification Result
`make up`	Deploys cluster, creates S3 buckets, builds container, and applies manifests in ≤3 commands.	Automated clean bootstrap
`make down`	Tears down deployments, terminates pods, stops S3, and cleans local volumes.	Zero resource leakage
`make test`	Executes pytest unit test suite covering service endpoints and schema logic.	5 passed in 10.81s
`make lint`	Runs flake8 linter with .flake8 configuration across all python modules.	0 warnings / 0 errors
`make train`	Executes student knowledge distillation pipeline with SUBSET override support.	Loss: 3.699 → 1.550

AI Usage Transparency (AI_USE.md):

In accordance with project guidelines, all AI assistance is rigorously documented in `AI_USE.md`. AI was utilized as a pair programmer for PyTorch backward profiling diagnosis, drafting Kubernetes manifests, and formulating regex patterns. Every line of AI-suggested code was independently tested, profiled, and verified by human engineering.

7. Task 7: Performance, Cost Modeling, and SRE Review

Empirical Load Test Results (Locust):

A 2-minute headless Locust benchmark (`locust -f tools/locustfile.py --headless --users 4 --spawn-rate 4 --run-time 2m`) was executed against the deployment on port 8000. Measured results (`results/loadtest_stats.csv`):
• **Total Requests:** 3,264 requests | **Failures:** 0 (**Error Rate: 0.0%**)
• **Throughput: 27.23 requests/second** | **Average Latency:** 146.63 ms
• **Latency Distribution:** Min: 58.71 ms | **p50 (Median): 130 ms** | **p95: 250 ms** | Max: 622.61 ms.
The measured p95 latency of 250 ms is **20x faster** than the 5,000 ms grading ceiling.

Step-by-Step Cloud Cost Modeling:

Using the pricing parameters from `data/rate_card.json`:
• `vcpu_hour`: ■3.50 INR | `gib_memory_hour`: ■0.45 INR.
• Billing rule: *"A replica is billed for its CPU limit and memory limit for every hour it runs, whether or not it is busy."*
1. **Hourly Cost Per Replica:**
   $\text{Cost}_{\text{replica}} = (1.0\text{ vCPU} \times ■3.50) + (1.0\text{ GiB} \times ■0.45) = ■3.95\text{ INR/hour}$.
2. **Hourly Cost of Running Cluster (2 Active Replicas):**
   $\text{Cost}_{\text{cluster}} = 2 \times ■3.95 = ■7.90\text{ INR/hour}$.
3. **Requests Served Per Hour:**
   At measured throughput $R = 27.23\text{ req/s}$: $\text{Reqs}_{\text{hour}} = 27.23 \times 3,600 = 98,028\text{ reqs/hour}$.
4. **Cost per 1,000 Requests:**
   $\text{Cost}_{1000} = \left(\frac{■7.90}{98,028}\right) \times 1,000 = \mathbf{■0.0806\text{ INR}}\text{ (~8.06 paise per 1,000 requests)}$.
• Under baseline 10 req/s load: $\text{Cost}_{1000} = \left(\frac{■7.90}{36,000}\right) \times 1,000 = ■0.2194\text{ INR}$.

Service Level Objective (SLO) & Error Budget:

• **SLO Definition:** "99.0% of requests to /parse must succeed with HTTP 200 in under 5,000 ms."
• **Evaluation Window:** One 2-minute Locust load-test run (total 3,264 requests).
• **Error Budget (in Requests):** $\text{Budget} = \lfloor 3,264 \times (1 - 0.99) \rfloor = \mathbf{32\text{ requests}}$.
• **Error Budget Consumed: 0 requests consumed** (0 failures observed, 100% success rate). 100% of error budget remaining.

Kubernetes Pod Resource Consumption (kubectl top pods):

During peak 4-user concurrency, Pod CPU utilization averaged **310m (31.0%)** per replica (trigger threshold for HPA is 70%). Memory utilization stabilized at **554 MiB (54.1%)** per replica, demonstrating that the 1 GiB limit comfortably accommodates the PyTorch runtime, model weights, and request buffers without memory fragmentation.

SRE Production Review Table

Comprehensive review across all 6 production dimensions with exact measured values:

Component / Concern	Evidence (Measured Number)	Limitation Found	Improvement Proposed
Data and storage	1,792 commands processed; 1,717 cleaned records (SHA-256: b6e8e5dd...)	SeaweedFS volume.max=8 default saturated on local node.	Configure -volume.max=50 and distributed volume servers across disk mounts.
Model quality	59.0% full frame acc, 97.0% JSON validity (teacher: 62%, 99%)	70M parameter capacity restricts multi-slot complex syntax extraction.	Implement token-level KL distillation over intermediate hidden layers.
Latency and throughput	p50: 130 ms, p95: 250 ms, Throughput: 27.23 req/s	CPU contention during concurrent PyTorch forward passes on 1 core.	Quantize model to INT8 via ONNX Runtime CPU execution provider.
Resource utilisation	Peak RSS: 554.00 MiB (45.9% headroom below 1024 MiB limit)	Static memory dominated by PyTorch autograd engine & vocab embeddings.	Prune unused token embeddings (vocab 50304 → 2000 active domain tokens).
Reliability and recovery	Pod recovery: 12.8s; 0 failures during rolling update (3,264 reqs)	Single-node cluster cannot survive host machine hardware failure.	Deploy multi-AZ cluster with topologySpreadConstraints and PDB.
Security and cost	6/6 NACL packets passed; ■0.0806 INR per 1,000 requests	Stateless NACLs require wide ephemeral port opening (1024–65535).	Adopt stateful Calico eBPF network policies and automated IAM rotation.

Synthesis: Measured Results vs. Operational Assumptions

To ensure academic rigor, measured empirical numbers are strictly distinguished from operational assumptions:

Dimension	Strict Measured Result (Empirical Evidence)	Operational Assumption (Modeled Parameter)
Model Performance	Full-frame accuracy: 59.0%; JSON validity: 97.0%; eval file results/student_validation.json	Assumed validation.jsonl is representative of the hidden 300-command test set.
System Memory	Peak RSS: 554.00 MiB; baseline: 458.18 MiB; command log results/memory_profile.txt	Assumed 1 GiB container memory limit matches standard micro cloud instances.
Cold-Start Latency	Median cold start: 9.26s across 3 runs; evidence in results/cold_start.txt	Assumed local S3 loopback bandwidth approximates 10 Gbps cloud internal VPC networking.
Cloud Service Cost	Throughput: 27.23 req/s; 98,028 reqs/hour; Locust results/loadtest_stats.csv	Assumed hypothetical rates: ■3.50/vCPU-hr, ■0.45/GiB-hr from rate_card.json.

References (Excluded from Page Limit)

1. Kim, Y., & Rush, A. M. (2016). Sequence-Level Knowledge Distillation. *Proceedings of EMNLP 2016*, Austin, Texas.
2. Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.
3. Bastianelli, E., et al. (2020). SLURP: A Spoken Language Understanding Resource Package. *Proceedings of EMNLP 2020*.
4. Biderman, S., et al. (2023). Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *ICML 2023*.
5. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.
6. Kubernetes Documentation (2024). Resource Metrics Pipeline, Probes, and Rolling Updates. https://kubernetes.io/docs/
7. SeaweedFS Documentation (2024). S3 API, Filer Architecture, and Volume Growth. https://github.com/seaweedfs/seaweedfs

Appendix A: End-to-End Build & Run Instructions

1. Clone & Bootstrap:
`git clone https://github.com/NavaneethNemapu/cloud-command-parser.git && cd cloud-command-parser`
2. Launch Cluster & Deploy:
`make up` (Starts SeaweedFS, creates buckets, downloads release weights, and launches inference service)
3. Run Verification Tests:
`make test && make lint` (Runs 5 unit tests and flake8 style checks)
4. Execute Load Test:
`locust -f tools/locustfile.py --headless --users 4 --spawn-rate 4 --run-time 2m --host http://localhost:8000 --csv results/loadtest`
5. Teardown:
`make down` (Terminates services and cleans local volume mounts)

Appendix B: Release Asset Metadata & SHA-256 Checksums

- `models/v1/model.sha256`: `95053ae63c454e1d5b57a8dc9cbdc5483304e61dea7b7ba10e4eee0e679e1452`
- `models/v2/model.sha256`: `9d8678e277dad0b4c5630b326a6178e3ae9cb292ec216c8207f41d15f1e2c47a`
- `datasets/v1/train.sha256`: `b6e8e5ddad44ee3e5c7bfa6b2fac230befc3b6c09ae80a29f4ce6597bea5aacf`
- **GitHub Release:** All trained safetensors weights, tokenizer vocabularies, and config manifests are published under tag `v1.0.0`.